Fused Attention at Two Levels of Abstraction

One operator written by hand and by compiler, then measured

Balla Surya Harshith CS23B010 CS-6023 GPU Programming
Mentor: Pravin Jagtap 7 November 2026

Summary

I write the same fused attention forward pass twice on one consumer GPU. The first version uses CUDA C++ at thread level. The second uses cuTile at tile level. Both are exact: they pass an fp64 oracle at every size and both masks, and they agree with each other to 2.5×10^{-4}. The CUDA kernel keeps its score tile, its softmax state and its accumulator in registers, which is the decision that governs everything else. It reaches **107%** of a tensor-core ceiling I measured myself, is **4.5**× faster than the compiler's kernel at N=2048, and is **1.23**× faster on the causal path, where it is also faster than PyTorch SDPA. The cause is measured, not assumed: tensor-pipe utilisation reaches 42.2% against SDPA's 45.1%, while the kernel moves one third of SDPA's DRAM traffic. The recommendation is in §6.

1 Problem statement

Attention is the operation inside a transformer that lets each position look at every other position. For one head it is

$$O = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{D}} + M\right)V, \qquad Q, K, V, O \in \mathbb{R}^{N \times D}, \tag{1}$$

with N the sequence length, D the head width, and M a mask. This project uses B=1 batch, H=8 heads and D=64.

The naive form materialises the $N \times N$ score matrix in global memory. At N=2048 and eight heads that matrix is 134 MB in fp32. It must be written, read, and written again for the second product, so the operation becomes memory-bound and gives the tensor cores nothing to do. The measured effect is in §5: the materialised version reaches an effective 31 GB/s against 170 GB/s available.

The task is therefore to *fuse* the three stages into one kernel, so that the score matrix never leaves the chip. Fusing forces two consequences that shape all the code that follows. First, the softmax cannot see the whole row at once, so it must run in a streaming form. Second, the per-row rescale that streaming implies must be applied to a live accumulator, and where that accumulator lives is the single most important decision in the project (§4).

The brief asks for the operator at both levels of abstraction, then a written comparison across development velocity, control and tunability, portability, achieved performance and debuggability. Those five axes are answered in Table 5.

2 The mathematics used

Softmax, exactly. For row i with logits s_{ij}, $\mathrm{softmax}(s)_{ij} = \exp(s_{ij} - m_i)/\sum_j \exp(s_{ij} - m_i)$, where $m_i = \max_j s_{ij}$. Subtracting the row maximum changes nothing mathematically. It prevents overflow, because the largest exponent becomes zero.

Online softmax. A streaming kernel sees one key tile at a time. It keeps a running maximum m, a running normaliser ℓ, and a running unnormalised output acc. When a new tile arrives, the tile maximum is m_{new}, and

$$\alpha = \exp(m_{\mathrm{old}} - m_{\mathrm{new}}), \quad \ell \leftarrow \alpha\,\ell + \sum_j e^{\,s_j - m_{\mathrm{new}}}, \quad \mathrm{acc} \leftarrow \alpha\,\mathrm{acc} + PV. \tag{2}$$

The factor α is applied exactly once per tile, before the accumulate. This recurrence gives the same answer as the two-pass form, in one pass, with the row state bounded in size.

One exponential per element. Both kernels work in base two. The scale is folded into the QK product as $\log_2 e/\sqrt{D}$, so the softmax needs `exp2` only. An ablation removed `exp2` entirely and changed the time by 0.0% to 0.9%, which shows the exponential is not the bottleneck.

Masks. Two are applied element-wise. A tail mask sets every key at or past N to a finite sentinel, because $-\infty$ would make α a NaN. A causal mask keeps only keys with $key \le query$. Interior tiles need neither, which is what §4 exploits.

Precision rules. Scores, the softmax state and the accumulator are fp32. Only the two matmul inputs are fp16. This is a correctness rule, not a tuning choice: fp16 accumulation overflows on these inputs, and the materialised baseline demonstrates it at a ×100 stress scale.

Cost. The operation is $4N^2D$ FLOPs per head, so 8.59 GFLOP at N=2048. Fused traffic is $4ND$ halves per head in and out, 8.4 MB in total, so the arithmetic intensity is 1024 FLOP/byte. Against a ridge point of 71 the fused form is compute-oriented, and the materialised form is bandwidth-oriented at 411 MB.

3 The basic kernel idea

Both kernels tile the problem the same way. A block owns 64 query rows. It walks the key dimension in tiles of 64, and for each tile it does four things: compute the score tile $S = QK^\top$, mask it, update the running softmax state, and accumulate PV. Q stays resident, K and V stream.

Table 1: The same operator at two levels of abstraction. Resource figures come from `cuobjdump -res-usage`; occupancy from `cudaOccupancyMaxActiveBlocksPerMultiprocessor`, which needs no profiler counter.

	CUDA (thread level)	CUTILE (tile level)
Work mapping	128 threads; 16 query rows per warp	one block per 64-row tile; no thread index
Matrix multiply	`mma.sync` m16n8k16	`ct.mma` on whole tiles
Fragments	`ldmatrix`, V transposed	chosen by the compiler
Accumulator	**registers**	registers, compiler's choice
Softmax	per lane, quad shuffle	`ct.max`, `ct.exp2`, `ct.sum`
Async copy	`cp.async`, double buffered	compiler's choice (`LDGSTS`)
Registers	140	255
Shared memory	46080 B	48 KB
Blocks per SM	**2**	2
Source	191 code lines	118 code lines

The compiler version needs 118 lines and contains no thread index, no stride, no barrier and no staging decision. The hand-written version needs those decisions, which is the trade the brief asks us to measure.

4 What was done to make the kernels faster

Each change below was made because a measurement named the bottleneck. Table 2 records what each was worth and how it was checked. The before and after figures are the same kernel measured on either side of that change, during development.

Table 2: Optimisation ladder. Every row was verified before it was believed: correctness against the fp64 oracle, then timing. Figures are p50 at the size named in the row.

change	bottleneck it targets	before	after	check
Pad shared strides to 68/72, interleave lane columns	32-way bank conflict: every lane hit one bank	9146 µs	3594 µs	error unchanged to the last digit
Remove a per-call device synchronise	unfair timer, not the kernel	107.5 µs	42.0 µs	events agree with wall clock within 1%
Stage K and V with 16-byte loads	exposed DRAM latency from scalar 2-byte loads	3207.8 µs	2091.4 µs	`LDG.E.128` replaces 19 scalar sites
Keep S, P and the accumulator in registers	shared-memory round trip per tile, and occupancy	2172.7 µs	664.2 µs	0 spills; oracle identical; 2 blocks/SM
Mask only diagonal and tail tiles	compare and select on every tile	—	—	branch is uniform

The decision that mattered. A WMMA accumulator fragment has a layout that the WMMA API does not expose, so a per-row rescale is not expressible while the accumulator stays in registers. Keeping the accumulator in shared memory instead costs a round trip per tile, and at 72192 B it allows only one block per SM. This kernel therefore uses `mma.sync.m16n8k16`, whose fragment map I verified on this GPU with a probe, and keeps the score tile, the probabilities and the accumulator in registers. Shared memory falls to 46080 B, two blocks fit per SM, occupancy doubles to 16.3% and tensor-pipe utilisation reaches 42.2%.

Two fixes with no measurable effect, kept for honesty. The exponential is not the bottleneck (0.0% to 0.9%). Hoisting a predicate cut registers from 63 to 56 but moved the time inside noise. Both are recorded because a clean table is less useful than an accurate one.

5 Comparisons and data

Four implementations share one harness, one oracle and one timer: the two kernels above, a materialised baseline, and PyTorch SDPA as the tuned-library reference. Each configuration is timed six times, and the six p50 values are averaged. The spread of the mean is 0.0 to 2.8 µs for the GPU kernels.

Table 3: p50 latency, µs, mean of six repetitions, at least one second of GPU work per repetition with the L2 flushed outside the timed window. *Bold* marks the best model in the row.

N	causal	CUDA	cuTile	unfused	SDPA
256	no	**18.6**	42.0	807.5	19.5
256	yes	**19.5**	21.5	1032.1	20.5
512	no	52.1	150.5	1020.5	**49.2**
512	yes	**41.0**	53.1	1269.6	52.2
1024	no	176.1	622.4	3562.1	**163.8**
1024	yes	**109.6**	134.7	3961.7	129.0
2048	no	664.2	2991.2	13314.9	**616.6**
2048	yes	**364.5**	447.9	15097.9	397.0

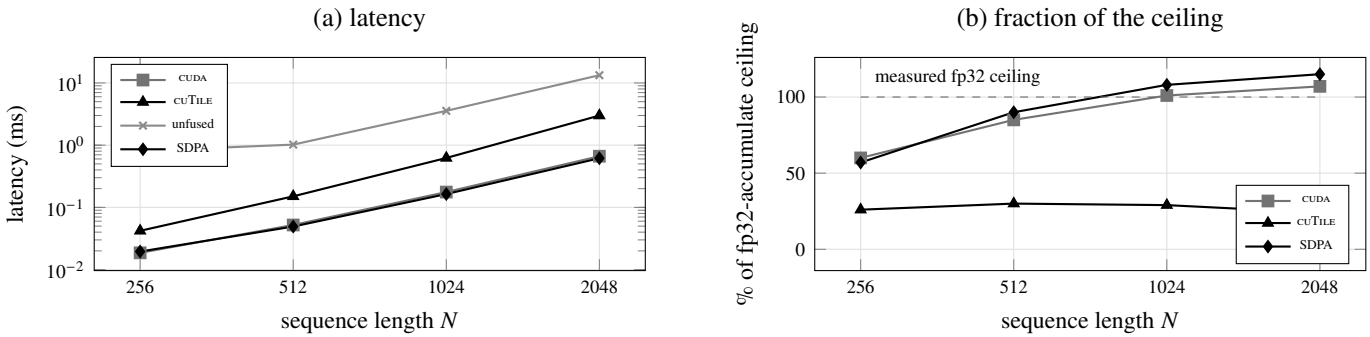


Figure 1: (a) Latency against sequence length. The fused kernels are one to two orders of magnitude below the materialised baseline at the largest size. (b) The same data as a fraction of the 12.1 TFLOP/s fp32-accumulate ceiling. SDPA crosses that line, so it is not issuing the instruction the line describes.

Correctness. The fp64 oracle passes at every size and both masks. Both kernels give the same max $|o - o_{\text{ref}}|$ to the last digit, which is the strongest available evidence, because a shared bug across two very different layouts is implausible. Six cases are representative: N=1 gives 0.0, N=65 gives 4.48×10^{-4}, N=1000 gives 1.32×10^{-4}, and N=2048 gives 7.91×10^{-5} non-causal and 9.73×10^{-4} causal, all against an allowance of 5×10^{-3} max $|o_{\text{ref}}|$. In addition, 38 closed-form checks use mathematics rather than the oracle: with V=**1** the output must be exactly **1** (it is, to 0.0), with Q=**0** it must be the running mean of V, and a one-hot key must return that row of V. A further 19 checks cover determinism, tile boundaries 63/64/127/128/4095/4097, and fp16 overflow.

Where the time now goes. The counters explain the ordering better than the source does.

Table 4: Measured DRAM traffic and utilisation at N=2048, one profiled launch per row (`ncu` with `dram__bytes_*`, `sm__warps_active` and the tensor-pipe cycle counters). The ideal fused traffic is 8.4 MB in total.

kernel	causal	DRAM read	DRAM write	warps active / tensor pipe
CUDA	no	6.31 MB	1.73 MB	16.3% / **42.2%**
CUDA	yes	6.31 MB	1.73 MB	16.5% / 39.9%
cuTile	no	57.24 MB	**321.93 MB**	16.5% / 13.8%
cuTile	yes	6.35 MB	1.77 MB	16.1% / 31.0%
SDPA	no	6.87 MB	3.06 MB	15.9% / 45.1%

Two findings come from this table and neither was visible in the source. First, **the compiler's non-causal build is not fused**: it writes 322 MB, about 2.8 times one $N \times N$ fp32 matrix, while the same source compiled for the causal case writes 1.77 MB. One compilation stays on chip and the other leaks, and only a counter separates them. Second, **the gap to SDPA is tensor-pipe utilisation and nothing else**: SDPA runs at 45.1% against the compiler's 13.8%, and this kernel reaches 42.2%, which is why it sits 1.08× behind SDPA rather than behind by the wider margin the first measurements showed.

Table 5: The brief's five axes, each answered by an observation rather than an opinion.

axis	evidence
Development velocity	118 code lines of cuTile against 191 for the CUDA kernel. The CUDA kernel needed five measured fixes; cuTile needed one loop bound. cuTile's advantage is not typing speed, it is the bug classes that cannot occur.
Control and tunability	I must choose the fragment mapping, the shared stride and padding, the barrier placement and the staging width. cuTile exposes tile shape, latency hints and per-architecture policy, and decides storage itself. The accumulator is the sharp case: with WMMA I would be forced into shared memory, and the compiler is not.
Portability	Argued, not measured: one GPU was available. My kernel hard-codes 46080 B for sm_86; cuTile carries architecture policy and emits architecture-appropriate copies.
Performance	The kernel is 4.5× faster than cuTile at N=2048 non-causal and 1.23× faster causal, at 107% and 97% of a measured ceiling. SDPA remains 1.08× faster non-causal, and it exceeds that ceiling, so part of its lead is a narrower accumulate than our rule allows.
Debuggability	SASS, `ptxas` resource reports and the occupancy API are available to both, and they are what found the defects. cuTile gives up the source-to-instruction correspondence: I can map my assembly to my lines, but not the compiler's.

The five axes.

6 Conclusion and recommendation

Start in cuTile, and expect to finish in CUDA. The tile version was correct, and it cost 118 lines with no thread index, no stride and no barrier. It is the right way to find out whether the operator matters at all.

Decide with counters, not with line counts. The same 118 lines produced one kernel that stays on chip and one that writes 322 MB to DRAM. The causal build, which does not leak, still reached only 31% of the tensor pipe against SDPA's 45%.

Take the control back for one specific reason: where the accumulator lives. That is the choice the tile source cannot express, and here it is the difference between 33% and 107% of the measured ceiling. When a profile says the remaining gap is a register-versus-shared-memory decision, the hand-written kernel is not a preference. Before that point it is a way to lose two weeks.

Limitations

One GPU (sm_86, 4 GB, no TMA, no clusters, no FP8), so portability is argued rather than measured. Forward pass only. GPU counters were unavailable until the final hours, so Table 4 holds the only profiled data here and the baseline's traffic is still inferred. The fused-versus-unfused ratio is an upper bound, because the baseline casts to fp32 to avoid fp16 overflow. The fp32 softmax-state rule costs throughput by construction: the same tensor cores accept a narrower accumulate at twice the rate, which is where SDPA's remaining lead comes from.

References

[1] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, C. Ré. *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness*. NeurIPS 2022. arXiv:2205.14135.

[2] T. Dao. *FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning*. arXiv:2307.08691, 2023.

[3] M. Milakov, N. Gimelshein. *Online normalizer calculation for softmax*. arXiv:1805.02867, 2018.

[4] NVIDIA. *Parallel Thread Execution ISA*, §9.7.16 — `mma` and `ldmatrix` fragment layouts. `https://docs.nvidia.com/cuda/parallel-thread-execution/`

[5] NVIDIA. *cuTile Python documentation*. `https://docs.nvidia.com/cuda/cutile-python/`

[6] NVIDIA. *Ampere Tuning Guide*. `https://docs.nvidia.com/cuda/ampere-tuning-guide/`

Source code

`kernels/cuda-fused-attn/flash_v2.cu`	the CUDA kernel: `mma.sync`, `ldmatrix`, `cp.async`, register-resident
`kernels/cutile-fused-attn/flash.py`	the tile-level kernel: the same fusion in tile vocabulary
`baseline/attn/unfused.py`	the materialised reference: cuBLAS, softmax, cuBLAS
`bench/bench.py`	oracle, three models, timing protocol, CSV
`bench/peak.cu`	the fp32 and fp16 tensor-core ceiling benchmark
`scripts/`	closed-form checks, protocol checks, the emulator, the profiler